Received: Added at production | Revised: Added at production | Accepted: Added at production

DOI: xxx/xxxx

RESEARCH ARTICLE

RAS-SDNCloudSim : Integrating Bandwidth-Latency-Aware Routing into SDNCloudSim Framework for Advanced Resource Allocation Scheduling Management

Nadia QOUDHADH*[1] | Mostapha ZBAKH[2] | Yassir SAMADI[3] | Claude TADONKI[4]

[1]National School of Computer Science and Systems Analysis, Mohamed V University in Rabat, Rabat, Morocco,
Email: nadia_qoudhadh@um5.ac.ma

[2]National School of Computer Science and Systems Analysis, Mohamed V University in Rabat, Rabat, Morocco,
Email: mostapha.zbakh@ensias.um5.ac.ma

[3]Faculty of Science of Tetouan, Abdelmalek Essaadi University, Tetouan, Morocco ,
Email: y.samadi@uae.ac.ma

[4]Centre de Recherche en Informatique (CRI), Mines ParisTech–PSL, Paris, France ,
Email: claude.tadonki@mines-paristech.fr

Correspondence
Corresponding author: Nadia QOUDHADH,
Email: nadia_qoudhadh@um5.ac.ma

Abstract

Cloud computing is now the backbone of the advanced information technology infrastructure, where appropriate resource scheduling is necessary to achieve efficiency in performance and cost. Although many resource scheduling algorithms have been designed, few of them have been tested in real-world network conditions, especially in Software-Defined Networks (SDN), where the interplay between routing decisions and resource allocation remains poorly understood.

To address this gap, this paper proposes a new routing strategy that dynamically computes the most suitable network path by jointly considering bandwidth availability and end-to-end latency. Called BLA (Bandwidth-Latency-Aware). This contribution is embedded within a broader extension of SDNCloudSim, a Software-Defined Networks simulation framework, which we enhance with a comprehensive Resource Allocation Scheduling (RAS) Management module. The proposed RAS-SDNCloudSim framework includes complementary advancements that work in synergy with BLA: (i) dynamic VM allocation policies (including MFF - Most Full First, LWFF Least Weast Fast First, and LFF Least Full First), enabling customizable resource management and load balancing across hosts; (ii) multiple workload scheduling policies (User defined priority, Shortest Job First SJF, and Particle Swarm Optimization PSO), allowing meaningful comparison of workload-level strategies; — The simulation results show that the proposed BLA strategy reduces the average packet delay by 12.85%, while the reduction exceeds 20% during peak times. The above-mentioned results place the proposed simulation framework in the position of being able to simulate and evaluate advanced simulation algorithms in realistic cloud-SDN systems.

KEYWORDS
Cloud Computing, Algorithm Scheduling, Bandwidth and Latency Management, CloudSimSDN, Network Optimization.

1 | INTRODUCTION

The IT industry has been significantly transformed by the emergence of cloud computing. Indeed, it offers elastic, scalable, and efficient service utility, thereby significantly reducing costs associated with hardware infrastructure and human resources. As a result, organizations can now dynamically allocate resources based on demand without facing over-provisioning or under-utilization. The deployment of services in cloud computing is broadly impacted by the elasticity, cost-effectiveness, and scalability of resources[1].

Resource Allocation and Scheduling (RAS) is a fundamental research area in cloud computing, where decisions have direct effects on system performance and cost[2]. RAS plays critical role in ensuring efficient resource utilization, reducing energy consumption, and maximizing profits for cloud service providers. Tinghuai Ma et al.[2] have classified the RAS problem into five primary focus areas, as illustrated in Figure 1 :

Abbreviations: BLA, Bandwidth-Latency Aware; SDN, Software-Defined Network; RAS, Resource Allocation Scheduling.

 © 2026 Copyright Holder Name

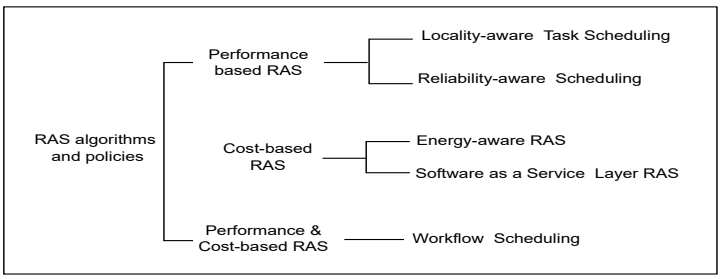

FIGURE 1 A taxonomy of RAS algorithms and policies[2]

Figure 1 above illustrates the five focus areas of RAS: (1) locality-aware task scheduling; (2) reliability-aware scheduling; (3) energy-aware RAS; (4) SaaS layer RAS; and (5) workflow scheduling. These focus areas are classified into three main categories: performance-based RAS, cost-based RAS and combined performance and cost-based RAS[2].

Testing different scenarios on live physical systems to maximize performance while minimizing cost is often too expensive, complex, time-consuming, and resource-intensive. To handle these challenges, simulation tools provide viable and cost-effective alternatives that help researchers to develop, refine, and test their approaches in controlled and repeatable conditions. Furthermore, such tools facilitate in-depth analysis of performance metrics about proposed solutions before their real-world deployment. A variety of simulation tools are used today in academic and practical research for modeling various aspects of cloud infrastructure and energy efficiency. Among such notable and widely adopted frameworks, CloudSim[3] stands out. Over the years, several extensions and frameworks have been developed on top of CloudSim. Each targeting specific areas such as WF-CloudSim for workflow scheduling, basic SDNCloudSim including network aspects, container management and among many others. However, SDNCloudSim still lacks an integrated version that combines key modules for bandwidth and latency optimization, VM allocation policies, workflow scheduling, and result analysis. In light of these limitations, the contribution of this work is to introduce an integrated SDNCloudSim combining bandwidth-latency-aware routing and advanced RAS management.

The remainder of this paper is organized as follows: Section II highlights the background and related works, outlining the existing solutions and their limitations. Section III describes our extended framework, RAS-SDNCloudSim, structured into two subsections

–

: the new bandwidth-latency-aware policy, VM allocation strategies, and workflow scheduling policies. Section IV reveals the findings by presenting an analysis of the proposed improvements in the paper, while Section V concludes the paper with possible future directions of related research studies.

–

2 | BACKGROUND AND RESEARCH CHALLENGES

This section introduces the CloudSim Toolkit framework and the extensions developed in previous simulation studies for the cloud computing environment, covering areas such as resource management, networks, containers, data placement and security.

2.1 | CloudSim Toolkit

CloudSim is recognized as the most prominent, advanced, and widely used simulation framework[3], which is characterized by the following properties:

Flexibility and Extensibility: is developed in a way that it can simulate various cloud infrastructure models, including data-centers, virtual machines, hosts, and resource allocation policies, which can facilitate various cloud-related research studies[4].

Scalability: supports different-scale simulations, ranging from smaller to larger topologies. This scalability enables realistic simulation of cloud system performance under varying loads. Giving information on resource allocation and load balancing[5].

Customization and Extensibility: due to its open-source model, it allows for the implementation and development of custom algorithms. Or the integration with supporting tools. This helps in simulating a specific algorithm related to scheduling, networks, or resource management[6].

Realistic Cloud Resource Simulation: supports real cloud resource simulation. From the physical level to the virtual one. This is a very important facet of cloud simulations, especially when it comes to cloud resource allocation and cloud energy efficiency, as indicated in a paper by Salem et al.[7].

CloudSim is a discrete-event simulation framework that consists of three layers as illustrated in Figure 2[8], each serving a specific purpose:

1. Core Simulation Engine: This level is responsible for cloud simulation events and entities, such as entity creation, scheduling, and processing. Entities here include hosts, virtual machines, brokers, and associated components.

2. CloudSim Layer: This layer captures the behavior of the cloud infrastructure. The network topology specifies the connections between cloud-related resources, such as virtual machines, brokers, and data centers, as well as resource allocation processes based on policies, scalability, and elasticity.

3. User code: In this tier users can customize the experimental setups. The user broker applies customized policies regarding resource management and task scheduling.

CloudSim has a modular and extensible architecture based on the separation of concerns principle. Users define the requirements, while CloudSim handles the simulation mechanism. This design promotes flexibility, reusability, and the ability to deal with multiple models of cloud computing by distinguishing the different levels of CloudSim architecture, as shown in Figure 2, proposed by Calheiros R et al.[8].

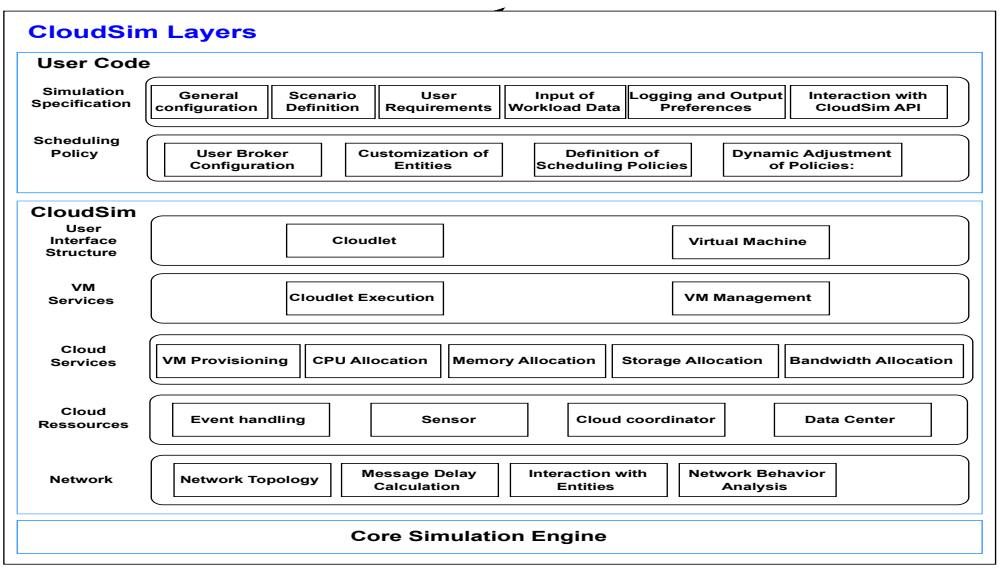

FIGURE 2 CloudSim architecture layers[8]

2.2 | Related Works

A variety of extensions have been developed for CloudSim to overcome various challenges. In the section below, we describe the contributions and shortcomings of related works, thereby opening the avenue for the present study.

CloudSimSDN, presented by authors Jungmin et al.[9] is an extension of the base CloudSim framework with the integration of Software Defined Networking (SDN) concept. It enables both physical and virtual network simulations controlled by SDN controllers and analyzes how networks impact system performance. However, despite the above-mentioned strengths, it lacks fine-grained and dynamic control abilities concerning complex workloads as well as advanced scheduling strategies.

WorkflowSim, developed by W. Chen et al.[10] focuses more on simulating scientific workflow applications in a distributed setup. This model takes into consideration the dependencies between tasks to increase the realism of simulated applications. Unfortunately, the model does not have the capability to simulate the network infrastructure, thereby making it impossible to derive information related to latency and bandwidth utilized by the workflow application. This becomes one of the key factors in our proposed method.

CloudSim Plus, as enhanced by M. C. Silva et al.[11], improves the modularity and scalability characteristics in CloudSim. By providing an extensible and well-structured simulation toolkit. While it integrates new policies and models, it remains focused on compute and resource management. Which reduces its suitability for scenarios where network behavior significantly influences system performance.

IoTSim[12] is an extension of CloudSim for Internet of Things (IoT) applications. It models the interaction between IoT devices and cloud infrastructure. IoTSim is useful for simulating latency and edge cloud resource allocation. However, it does not capture the network layer and lacks the programmability and path control features that SDN-based frameworks offer.

ContainerCloudSim, in its advanced form developed by N. Saleh et al.[13] simulates containerized environments. The authors identify three main areas of existing shortcomings in the ContainerCloudSim model. Firstly, they introduce a feature allowing data centers to host both containers and virtual machines (VMs). Secondly, they develop a dynamic simulation approach. Cloud tasks (cloudlets) are dynamically scheduled, instead of all being submitted at the beginning. This approach mimics real-world scenarios where tasks unpredictably arrive. Thirdly, they have incorporated interdependent task simulation using the concept of Directed Acyclic Graphs (DAGs), hence making their simulation of containers even more realistic. However, it is worth noting that the simulation still assumes a static opaque network. That limits the evaluation of policies that depend on real-time network metrics. A gap our work aims to address.

DFMCloudSim[14] is used to extend the features of the CloudSim simulator to support the concepts of data fragmentation and migration[15] in cloud-based applications. DFMCloudSim fragments a large amount of data and migrates only the required fragments, reducing transfer overhead. The tool simulates geographically distributed data centers, enabling users to test strategies before real-world deployment, with support for formats such as plain text files.

Moreover, recent research focuses on authentication and key agreement between controllers and edge nodes[16]. These efforts extended SDN cloud environments by developing secure and lightweight communication frameworks. While these works bring enhanced trust and privacy in emerging cloud-edge networks, none of them tackle dynamic bandwidth and latency-aware scheduling, which is the core focus of the present study.

Then, Anand et al.[17] offer a comprehensive overview of SDN security mechanisms against DoS and DDoS attacks, which include FloodDefender, SDN-Guard, and the entropy-based detection strategy. Their results show how to integrate security. Performance optimization is an open challenge in SDN environments, an aspect that RAS-SDNCloudSim tries to address by integration of bandwidth and latency-aware policy. These tools illustrate the advancement of CloudSim-based simulations. Each extension addresses specific challenges such as workflow modeling, IoT integration, data migration, security, and container orchestration. Yet, they present common limitations: a lack of detailed network awareness. The basic parameters, such as bandwidth, latency, or dynamic link selection policies, remain unaddressed. The design of RAS-SDNCloudSim aims to extend the functionalities of CloudSimSDN in a consistent manner, requiring fast communication between microservices. This involves overcoming the current limitations in cloud simulation tools to incorporate a scheduling mechanism that considers bandwidth and latency.

2.3 | Research Challenges

The design of RAS-SDNCloudsim is motivated by the following key challenges :

1. **(i) Network-Aware Resource Scheduling** : How can resource allocation and workload scheduling decisions simultaneously account for network-level constraints namely available bandwidth and end-to-end latency in an SDN-controlled cloud infrastructure?
2. **(ii) Dynamic and Multi-Criteria Path Selection:** How to dynamically select the optimal network path in a multi-hop SDN topology by jointly optimizing bandwidth availability, latency, and energy consumption, rather than relying on static or arbitrary default routing? The trade-off between maximizing throughput, minimizing end-to-end delay, and reducing energy consumption constitutes a multi-objective optimization problem that varies with real-time traffic conditions. Selecting a path with higher available bandwidth may reduce queuing delays but could increase energy consumption by activating

additional switches or links. Conversely, consolidating traffic onto fewer links saves energy but risks creating congestion and violating SLAs.

To address these challenges, we propose RAS-SDNCloudSim with a Bandwidth-Latency-Aware (BLA) routing policy, modular VM allocation and scheduling components.

2.4 | Our contribution

In this paper, we propose a new Bandwidth-Latency-Aware policy for selecting links. Unlike the default CloudSim SDN routing mechanism, which chooses network paths in a static or arbitrary manner without considering network performance parameters, the proposed BLA policy will evaluate all possible paths between a given source and destination based on the bandwidth and latency of those paths. The path that provides the best trade-off between available bandwidth and end-to-end latency is selected as the routing path to transmit the workload. As illustrated below in Figure 3, workloads are submitted through the cloud broker, scheduled by algorithm scheduling, then routed via BLA through the hierarchical network (core → aggregate → edge switches) to reach VMs hosted on edge servers.

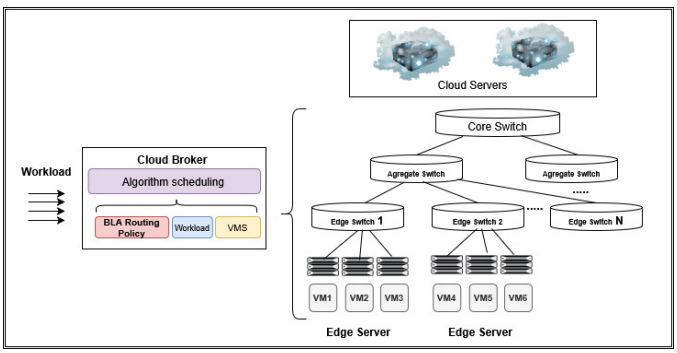

FIGURE 3 Cloud Scheduling environment[18]

3 | SYSTEM MODEL AND PROBLEM DEFINITION

This section defines the architectural and mathematical environment of the proposed RAS-SDNCloudSim framework, establishing the formal basis for the Bandwidth-Latency-Aware (BLA) routing strategy.

3.1 | System Model

Cloud computing environments rely on efficient network performance to support data-intensive workloads such as video streaming, big data analytics, microservices, and HPC. Bandwidth and latency are critical factors directly affecting data transfer rates and end-to-end QoS[19,1].

Bandwidth refers to the maximum data rate supported by a communication link (bits/s), while **throughput** denotes the effective rate achieved in practice, which is typically lower due to congestion, protocol overhead, and inefficiencies[19,20,21].

End-to-end latency is decomposed into four fundamental components. For any path p consisting of k hops:

$$L_{e2e}(p) = \sum_{i=1}^{k} \left(D_{\text{proc},i} + D_{\text{prop},i} + D_{\text{trans},i} + D_{\text{queue},i} \right), \tag{1}$$

where each component is defined as follows.

Processing delay (D_{proc}): the time required by a network device (sender, switch, or receiver) to process a packet before forwarding, including header parsing and routing table lookup.

Propagation delay (D_{prop}): the time for a signal to traverse the physical medium[19]:

$$D_{\text{prop}} = \frac{d \times n}{c}, \tag{2}$$

where d is the physical distance, $c = 3.0 \times 10^8$ m/s is the speed of light in vacuum, and n is the refractive index of the medium (e.g., $n \approx 1.5$ for optical fiber).

Transmission delay (D_{trans}): the time to push a packet onto the link[19]:

$$D_{\text{trans}} = \frac{S}{\eta B}, \tag{3}$$

where $\eta \in (0, 1]$ represents effective bandwidth efficiency under contention and protocol overhead.

Queuing delay (D_{queue}): the time packets spend waiting in buffers at switches due to congestion. Modeling each link as an $M/M/1$ queue[22]:

$$D_{\text{queue}} = \frac{\rho}{\mu(1-\rho)}, \tag{4}$$

where $\rho = \lambda/\mu$ is the link utilization and λ is the packet arrival rate.

The cumulative effect of these delay components is illustrated in Figure 4, which highlights the role of switch processing delay as a key performance factor[18,23].

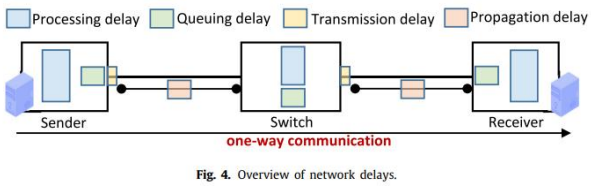

FIGURE 4 Overview of network delays in SDN environments[18].

The physical infrastructure is modeled as a hierarchical directed graph:

$$G = (N, L), \tag{5}$$

where $N = H \cup S$ is the set of nodes comprising physical hosts $H = \{h_1, \ldots, h_n\}$ and SDN switches $S = \{s_1, \ldots, s_m\}$ organized into three layers — **Core** (C), **Aggregate** (A), and **Edge** (E) — and $L = \{l_1, \ldots, l_k\}$ is the set of communication links.

Each link $l \in L$ is characterized by the tuple (B_l, d_l, n_l, S_l), where B_l is the total bandwidth capacity, d_l is the physical distance between endpoints, n_l is the refractive index of the medium, and S_l is the average packet size .

To ensure a realistic simulation of dynamic network behavior, the model incorporates:

- **Network state sensing**: The system tracks real-time link utilization $\rho_l = \frac{\text{usedBW}_l}{B_l}$, where usedBW_l is the currently allocated bandwidth. To prevent numerical instabilities under saturation, ρ_l is capped at 0.99.
- **Congestion modeling**: To avoid numerical instabilities during peak traffic, the link utilization is capped at $\rho = 0.99$.
- **Service rate formulation**: Each network link is modeled as an $M/M/1$ queue, where the service rate μ is defined as $\mu = \frac{B}{S}$, with B being the link bandwidth (bits/s) and S the average packet size (bits)[22].

3.2 | Problem Formulation

The research addresses the lack of synergy between computational scheduling and network-aware orchestration. Standard approaches employ a Select-First-Link (SFL) policy, which selects the first available path $p_1 \in P$ regardless of network congestion.

The challenge is formalized as: find an optimal path P^* that satisfies bandwidth requirements while minimizing end-to-end latency L_{e2e}.

The latency for any path $p \in P$ is the sum of four delay components :

$$L_{e2e}(p) = \sum_{i=1}^{k} (D_{proc,i} + D_{prop,i} + D_{trans,i} + D_{queue,i}) \tag{6}$$

3.3 | Objective Function

The BLA policy aims to minimize end-to-end latency while satisfying bandwidth constraints:

$$P^* = \arg\min_{p \in P} \; L_{e2e}(p) \quad \text{s.t.} \quad \min_{l \in p} B_l^{avail} \geq B_{req} \tag{7}$$

where $L_{e2e}(p)$ is the total latency of path p (sum of processing, propagation, transmission, and queuing delays), and B_{req} is the workload's bandwidth requirement.

3.4 | The BLA Proposed Strategy

The Bandwidth-Latency-Aware (BLA) strategy represents a paradigm shift from static to dynamic orchestration. Unlike the SFL policy, BLA performs a multi-criteria optimization as described in Algorithm 1:

Algorithm 1 Bandwidth-Latency-Aware (BLA)-Path Selection Procedure

```
Input:  f: workload flow, S: packet size, h_s, h_d: source and destination hosts
Output: P*: optimal network path

P <- Identify set of all possible network paths between h_s and h_d

for each path p_j in P do
    B_path(p_j) <- min(availableBandwidth) of all links l in p_j
    if B_path(p_j) >= S_req then
        D_proc <- Retrieve CPU processing delay for nodes in p_j
        D_queue <- rho / (mu * (1 - rho))
        L_e2e(p_j) <- D_proc + D_trans + D_queue
    end if
end for

P* <- Select p_j that minimizes L_e2e(p_j) while maximizing B_path(p_j)
Route workload flow f through selected optimal path P*
return P*
```

3.5 | System Workflow

Given a workload, the BLA policy is executed as follows:

- All possible network paths between the source and destination are identified.

- The minimum available bandwidth along all the identified paths is calculated.
- The end-to-end latency of all identified paths is estimated by summing up processing delay, propagation delay, transmission delay, and queuing delay.
- A composite cost is calculated by combining bandwidth availability and latency.
- The path with optimal trade-off between low latency and high available bandwidth is identified for packet transmission.

The mechanism of workload processing is systematically represented in the sequence diagram below and comprehensively described thereafter.

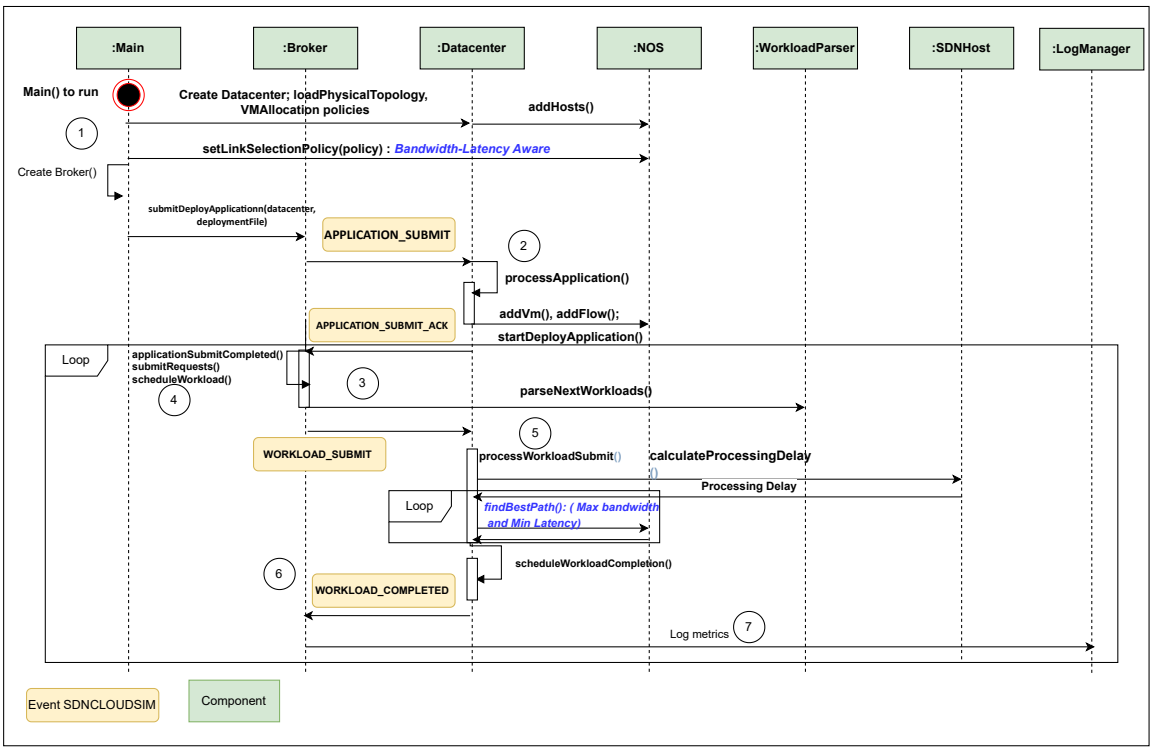

FIGURE 5 Workflow of application deployment and workload execution

Figure 5 illustrates the complete workflow of application deployment. The process begins in the initialization phase, triggered by the main() method. Step 1 configures the simulation environment. The broker is instantiated, and the physical topology is loaded. Then allocation and link-selection policies are established. This configuration ensures that subsequent resource provisioning and routing decisions are aligned with the chosen optimization strategies.

Still in Step 1, the submitDeployApplicationn() method is called to provide the broker with the deployment file. It defines the virtual topology. The broker then sends an APPLICATION_SUBMIT event to the datacenter in Step 2 to trigger the processApplication() method. In this step, virtual machines (VMs) and network flows are parsed and added to the Network Operating System (NOS), enabling application deployment.

After the deployment process. In Step 3, the datacenter sends back an APPLICATION_SUBMIT_ACK event. Containing critical mappings such as VM name-to-ID, flow ID assignments, and bandwidth configurations. Upon receiving this acknowledgment, the broker invokes applicationSubmitCompleted(). Which initializes workload parsers and begins reading the workload files using the parseNextWorkloads() method in Step 4.

The parsed workloads are submitted to the datacenter through WORKLOAD_SUBMIT event. In Step 5, the datacenter processes the request. By identifying VMs and hosts source and destination. Calculating CPU processing delay (calculateProcessingDelay()). Then, the optimal network path is selected by using the preconfigured link selection policy (findBestPath()).

In Step 6, all delays (processing, propagation, transmission) are calculated. The WORKLOAD_COMPLETED event is then scheduled and passed back to the broker.

Next, the broker obtains key performance metrics such as processing time, propagation, transmission, queuing delays, and total latency. All these details are encapsulated through a RequestMetrics object, which is stored along with the LogManager in Step 7. The design provides a clean separation of concerns. Between parsing, deployment, execution, and logging. Besides supporting dynamic scheduling and fine-grained performance monitoring.

3.6 | RAS-SDNCloudSim Architecture

The proposed RAS-SDNCloudSim focuses on the dynamic routing stategy, multi-objective resource allocation and monitoring process. This work combines the aforementioned issues into four key components:

- Bandwidth and Latency Aware (BLA) Rooting Strategy.
- Advanced VM Allocation Policies: Various heuristic algorithms such as Least Waste Fast First (LWFF), Most Full First (MFF), Least Full First (LFF), and First-Come-First-Served (FCFS) are implemented in the RASSDNCloudSim. The module is upgraded with different algorithms, increasing its ability to handle resource management in different cloud workloads.
- Workflow Scheduling: The workload allocation package is improved by adding various workflow scheduling techniques, such as User Defined Priority (Priority), Shortest Job First (SJF) and Particle Swarm Optimization (PSO).
- Logs and Monitoring: Include LogManager and LogMonitor modules. It tracks metrics for CPU, RAM, bandwidth, delays, energy consumption, and SLA violations during the simulation.

The package diagram in Figure 6 shows how the RAS-SDNCloudSim architecture. It is organized into modules. This diagram highlights the updated and newly added packages in SDNCloudSim. The project is organized as several packages. Where each

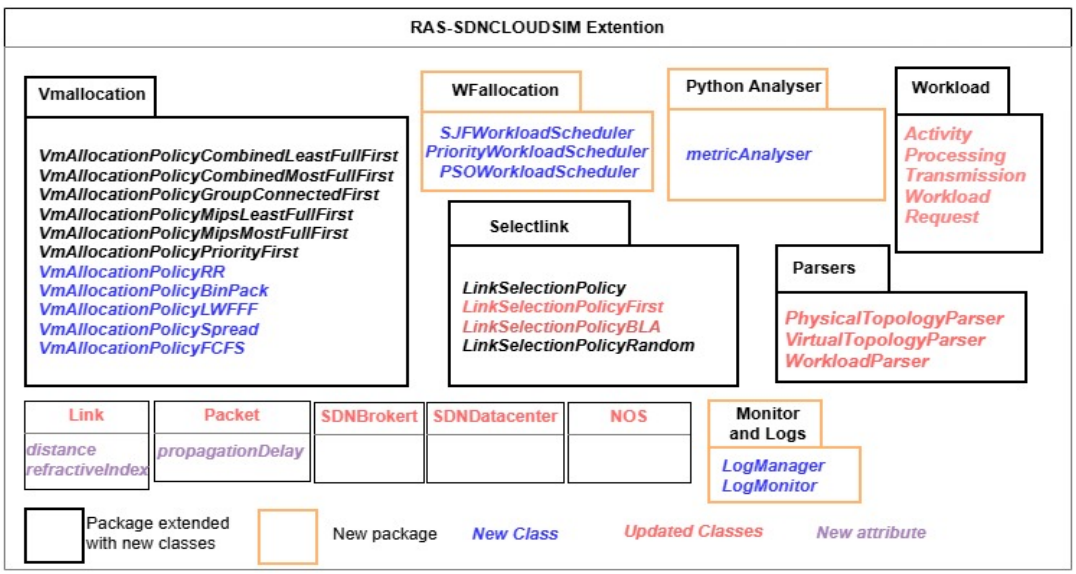

FIGURE 6 RAS-SDNCloudSim Extension package diagram

one encapsulates specific functionalities. The SDNCloudsim structure is extended by updating or adding new classes. The following overview below shows these packages with an emphasis on newly added classes and their roles:

VmAllocation: This package includes a distinct set of VM placement strategies tailored for different objectives. It is updated with different traditional and custom heuristic policies.

WFallocation: This package contains workload-level scheduling policies.

SelectLink: This module provides routing policies for link selection in SDN environments. It includes both basic and advanced strategies.

Parsers: Custom parsers aid in accurate and flexible data parsing. These are used for advanced topology and workload specifications.

Workload: This package models execution behavior of distributed services and dynamic applications. By using a fine-grained structure: workload, activity, request, processing, and transmission. All these components cover computation and communication workloads in microservices.

Monitor and Logs: The LogMonitor extends the base class SimEntity in order to monitor events occurring on fixed time interval in the simulation environment. It gathers statistics related to hosts, virtual machines (CPU, RAM) as well as network links (Bandwidth, Latency and Delay) from the simulation environment.

The collected data is buffered and processed in the form of CSV logs through the LogManager tool. It provides a format for writing all the collected measurement results to fixed output files, including real-time and global information like VM allocations, workflow scheduling policies, among others. When all workloads are completed, the monitoring process stops automatically. This is managed through the STOP_MONITORING event. By using the STOP_MONITORING event which triggers a graceful shutdown of the data collection mechanism.

Link class: is improved to automatic calculation of the propagation delay depending on the distance and the refractive index, allowing the simulation procedure for network behaviors, including congestions, in the SDN cloud environment.

Packet: extended to track propagation delay and failure handling. Moreover, it accesses the source host through the Network Operating System (NOS) class. Fine-grained tracing of network delays and packet failure is enabled.

SDNDatacenter: updated to support dynamic workload submission, link-aware scheduling, and accurate latency tracking (processing, propagation and transmission).

NetworkOperatingSystem: orchestrates VM deployment, flow configuration, and dynamic channel management in an SDN-based cloud simulation. Enhanced with bandwidth-latency-aware routing, topology construction, and dynamic monitoring of host, VM, and link utilization. Support for logging and monitoring is enabled via LogManager and LogMonitor, with real-time control of simulation events.

Python Analyser: this package contains metricAnalyser script. It loads CSV metrics log files such as energy consumption, delay, and resource utilization. It then generates comparative graphs. The result provides clear visual insights into energy usage, packet delays, and host metrics.

3.7 | Workflow Scheduling Policies

In cloud computing, workflow scheduling can be categorized into simple and scientific workflows. Simple workflows consist of sequences of tasks with minimal dependencies. While scientific workflows (Montage, LIGO, CyberShake, SIPHT, and Epigenomics) involve complex, data-intensive computations in fields like astronomy, seismology, and bioinformatics. These scientific workflows demand high processing power and benefit from cloud computing on-demand resource provisioning. Generally, workflow scheduling can be classified into user-level (handling service provisioning for users) and system level (managing cloud data center resources). Furthermore, scheduling methods can be static (fixed task-resource mapping) or dynamic (adapting to task arrivals in real-time). Other classifications include centralized vs. distributed, preemptive vs. non-preemptive, and online vs. offline scheduling[24]. Scientific workflows are often modeled as directed acyclic graphs (DAGs). It consist of interdependent tasks executed in a specific order. In a DAG, nodes represent tasks, and directed edges represent dependencies between tasks. To avoid circular dependencies and enable efficient scheduling tasks are executed in a specific order[25]. The core objective of workflow scheduling is to assign these interdependent tasks to available computational resources (e.g., virtual machines, CPUs, GPUs). Then, optimize key metrics such as makespan (total execution time), cost, energy consumption, reliability, load balancing, service-level agreement compliance, and security awareness. Cloud environments are inherently dynamic, and the diversity of cloud resources makes the scheduling process even more complex[26].

Regarding the scheduling of tasks in cloud infrastructure, there are predominantly two methods: heuristic techniques and metaheuristic techniques. The first one uses static criteria to allocate tasks to resources. These techniques include approaches such as Heterogeneous Earliest Finish Time (HEFT), Min-Min, Max-Min, and Round Robin. Metaheuristic techniques, on the other hand, use an advanced search process involving the fusion of various techniques based on evolutionary algorithm[27]. Like: Ant Colony Optimization (ACO), which is an ant-based process that applies the search strategies used by ants when foraging for food in search of task scheduling path optimizations; Particle Swarm Optimization (PSO), which allocates tasks to virtual

machines in an optimized process involving task positioning in the swarm search process; Genetic Algorithms (GA), which applies the concepts of selection and mutation in task/resource allocations; and Simulated Annealing (SA), which applies the concepts of annealing in the reduction of costs and time consumption[24].

Apart from the virtual machine allocation mechanisms, the RAS-SDNCloudSim module also supports a number of workflow scheduling mechanisms. The SJFWorkloadScheduler was chosen due to its simplicity in reducing the average task completion time. This algorithm favors short workflows, makespan, and resource utilization, especially in a scenario dominated by a number of small jobs. When it is compared to FCFS or Round Robin, the SJF offers better performance with low overhead, making it a practical and lightweight scheduling choice for simulation contexts[28].

We have also implemented the Particle Swarm Optimization (PSO) algorithm[27]. Which is a metaheuristic search algorithm that uses a population of candidate solutions, referred to as particles, whose behavior is based on social colony movement. The search for the solution is performed using the iterative process of updating candidate solutions based on individual experience and social intelligence, thereby effectively identifying near-optimal solutions for the complex nature of cloud infrastructure.

Additionally, we introduced User Defined Priority scheduling, allowing workflow submitters to assign priority levels to their workloads. High-priority workflows preempt lower-priority ones, ensuring critical business applications meet SLA requirements while maintaining fairness through priority aging mechanisms to prevent starvation.

Combining these workflow scheduling policies with VM allocation strategies offers a holistic approach to optimize the use of resources in the RAS-SDNCloudSim. The choice of the appropriate policy should correlate with the application requirements. Therefore, the approach can provide optimal usage of resources, fast execution times of workflows.

4 | EXPERIMENT AND DISCUSSION

In this section, we evaluate the benefits of the proposed extensions under realistic conditions based on several key performance metrics.

4.1 | Experimental Set-up and Architecture

5 | CONCLUSION AND PERSPECTIVES

Our paper introduces RAS-SDNCLOUDSIM, an extension of the CloudSimSDN platform that bridges a critical gap between computational scheduling and network-aware orchestration in cloud simulation. By integrating a Bandwidth-Latency-Aware (BLA) link selection policy, our framework enables fine-grained evaluation of resource allocation and routing strategies in microservice-based cloud environments. This work addresses the essential need for joint optimization of computational and network resources — an aspect often overlooked in existing CloudSim-based frameworks. We developed a modular experimental framework that incorporates detailed performance metrics, including delay components (processing, transmission, propagation, queuing), energy consumption, and comprehensive resource utilization (CPU, RAM, bandwidth), to thoroughly assess the effectiveness of the proposed BLA policy alongside complementary workload scheduling mechanisms.

Through simulation experiments using both physical and virtual topologies, we demonstrated that the proposed BLA policy substantially reduces end-to-end packet delays compared to the default Select-First-Link (SFL) approach, while maintaining stable energy consumption. This BLA module constitutes the core scientific contribution of this study, as it introduces a dynamic, multi-criteria routing mechanism that jointly optimizes computational and network resources—an aspect. Unlike existing approaches that rely on static link assumptions, the BLA mechanism adapts routing in real time based on bandwidth availability and latency feedback, thereby selecting the least congested and most efficient paths.

Quantitatively, the BLA mechanism achieves an average packet-delay reduction of approximately **12.85%** compared to SFL, with peaks exceeding **20%** for critical service communications . This consistent reduction directly translates into lower overall network latency, confirming the strong impact of bandwidth–latency–aware routing on Quality of Service (QoS) and responsiveness in SDN-enabled clouds.

In addition, we also implement, test and evaluate various workload scheduling strategies, including Shortest Job First (SJF), Hybrid SJF (SJFH), and Particle Swarm Optimization (PSO). The results indicate that PSO achieves the best trade-off in heterogeneous workloads, while simpler strategies such as SJF and SJFH provide greater stability with lower overhead in homogeneous environments.

These findings confirm an inherent trade-off between latency reduction and energy efficiency: while advanced mechanisms (e.g., BLA and PSO) significantly improve QoS and reduce delay, they may incur slightly higher computational costs. This emphasizes the need for adaptive, feedback-based policies capable of dynamically balancing performance objectives according to system constraints.

This flexible architecture opens new research perspectives, such as the simulation of microservice deployments based on containerized microservices for lightweight deployment scenarios (for example, based on Kubernetes). As well as the integration of advanced VM allocation policies (e.g., BestFit, LWFF, Energy-Aware). Additionally, "intelligent GA algorithms to optimize the scheduling process" can be leveraged.

Future work will extend

For the future, the work will compare the proposed RAS-SDNCloudSim simulator to existing state-of-the-art cloud and edge cloud simulators like iFogSim, EdgeCloudSim, in order to measure its extensibility to the hybrid cloud edge scenario. Moreover, the evaluation of larger and more complex topologies is necessary to further validate scalability and robustness. Various cloud security solutions such as isolation, access control[16] and threat detection[17,29,30] will enable a more realistic and comprehensive evaluation of cloud-native scheduling in dynamic, multi-tenant environments.

CAPTIONS AND ALT TEXT

Figure 1 Caption: A taxonomy of RAS algorithms and policies
Figure 1 Alt Text: Hierarchical diagram categorizing RAS algorithms into proactive and reactive policies with subtypes including load-aware, threshold-based, and hybrid strategies.

Figure 2 Caption: CloudSim Layer
Figure 2 Alt Text: Layered architecture of CloudSim simulation framework showing components from user level to datacenter, including VM, resource, and event management layers.

Figure 3 Caption: RAS-SDNCloudSim Extension package diagram
Figure 3 Alt Text: UML package diagram showing modules such as scheduling, SDN controller, and RAS integration in the RAS-SDNCloudSim simulation extension.

Figure 4 Caption: Overview of network Delays
Figure 4 Alt Text: Illustration showing types of network delays including queuing, transmission, propagation, and processing, across a virtual SD-WAN environment.

Figure 5 Caption: Workflow of application deployment and workload execution
Figure 5 Alt Text: Flowchart illustrating steps of application deployment, including service instantiation, VM placement, and cloudlet execution.

Figure 6 Caption: Physical and Virtual Topology
Figure 6 Alt Text: Diagram showing the mapping between physical and virtual network components including hosts, VMs, and SDN switches.

Figure 7 Caption: Energy consumption impacted by different Link Selection Policies
Figure 7 Alt Text: Bar chart comparing the maximum energy consumption (J) per host under different network link selection policies.

Figure 8 Caption: Packet Delay
Figure 8 Alt Text: Line plot showing packet delay variations over time under different routing strategies.

Figure 9 Caption: Average CPU Utilization per Host
Figure 9 Alt Text: Bar chart showing average CPU utilization for each physical host in the datacenter during the simulation.

Figure 10 Caption: Energy consumption
Figure 10 Alt Text: Bar chart comparing energy usage across different link selections in scenario 1.

Figure 11 Caption: Average Packet Delay Comparison – SFL vs. BLA with WD Scheduling Strategies
Figure 11 Alt Text: Comparative bar chart showing average packet delay for two routing policies (SFL and BLA) under Weighted Delay scheduling.

Figure 12 Caption: Average Packet Delay per Path - Comparison between SFL and BLA
Figure 12 Alt Text: Bar chart showing path-specific average delays under two routing strategies, highlighting the impact of link selection.

Figure 13 Caption: Average CPU Utilization per Host – Comparison of Scheduling Policies
Figure 13 Alt Text: Bar chart comparing host CPU utilization across various workload scheduling policies such as SJF, PSO, and Hybrid.

Figure 14 Caption: Impact of Scheduling and Link Selection Policies on Average Packet Delay
Figure 14 Alt Text: Combined chart comparing average packet delays across combinations of link selection and scheduling policies.

Figure 15 Caption: Average CPU Utilization per Host under BLA and SFL Link Policies (Default Scheduling)
Figure 15 Alt Text: Bar chart comparing CPU usage under two routing strategies with default scheduling across hosts.

Figure 16 Caption: Average CPU Utilization per Host under BLA and SFL Link Policies (PSO Scheduling)
Figure 16 Alt Text: CPU usage comparison across hosts under PSO scheduling with two routing strategies: BLA and SFL.

Figure 17 Caption: Average CPU Utilization per Host under BLA Routing with Different Scheduling Policies
Figure 17 Alt Text: Bar chart showing CPU utilization under BLA routing strategy with scheduling policies including default, SJF, and PSO.

AUTHOR CONTRIBUTIONS

AUTHOR CONTRIBUTIONS

Nadia QOUDHADH: Conceptualization, Methodology, Software, Writing – original draft, Visualization.
Mostapha Zbakh: Conceptualization, Formal analysis, Supervision, Validation, Resources, Writing - review - editing.
Yassir Samadi: Conceptualization, Formal analysis, Supervision, Resources, Validation, Writing - review - editing.
Claude Tadonkey: Conceptualization, Formal analysis, Supervision, Resources, Validation, Writing - review - editing.

FINANCIAL DISCLOSURE
No funding was received for the research, authorship, and/or publication of this article.

CONFLICT OF INTEREST
No potential conflict of interest was reported by the author(s).

SUPPORTING INFORMATION
The data that support the findings of this study are available from the corresponding author, NQ, upon reasonable request.

REFERENCES

1. Armbrust M, Fox A, Griffith R, et al. A view of cloud computing. *Commun ACM.* 2010;53(4):50–58. doi: 10.1145/1721654.1721672
2. Ma T, Chu Y, Zhao L, Ankhbayar O. Resource allocation and scheduling in cloud computing: policy and algorithm. *IETE Tech Rev.* 2014;31(1):4–16. doi: 10.1080/02564602.2014.890837
3. Barbierato E, Gribaudo M, Iacono M, Jakobik A. Exploiting CloudSim in a multiformalism modeling approach for cloud based systems. *Simul Model Pract Theory.* 2019;93:133–147. doi: 10.1016/j.simpat.2018.09.018
4. Kecskemeti G. DISSECT-CF: a simulator to foster energy-aware scheduling in infrastructure clouds. *Simul Model Pract Theory.* 2015;58:188–200. doi: 10.1016/j.simpat.2015.05.009
5. Thanapal P, Saleem M, Durai S. Energy saving offloading scheme for mobile cloud computing using CloudSim. *Int J Adv Intell Paradigms.* 2018;10(1–2):45–62. doi: 10.1504/IJAIP.2018.089488
6. Ranjan K, Sahoo G. Cloud computing simulation using CloudSim. *arXiv preprint.* 2014. arXiv:1403.6743.
7. Salem D, Haggag M, Mostafa M. Multi-core processors in cloud computing using CloudSim. *Int J Comput Appl.* 2016;137(8):31–34. doi: 10.5120/ijca2016908869
8. Calheiros R, Ranjan R, Beloglazov A, De Rose C, Buyya R. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. *Softw Pract Exp.* 2011;41(1):23–50. doi: 10.1002/spe.995
9. Son J, Dastjerdi A, Calheiros R, Ji X, Yoon Y, Buyya R. CloudSimSDN: modeling and simulation of software-defined cloud data centers. In: 2015:475–484
10. Chen W, Deelman E. WorkflowSim: a toolkit for simulating scientific workflows in distributed environments. In: 2012:1–8
11. Silva Filho M, Oliveira R, Monteiro C, Inacio P, Freire M. CloudSim Plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. In: 2017:400–406
12. Zeng X, Garg S, Strazdins P, Jayaraman P, Georgakopoulos D, Ranjan R. IOTSim: a simulator for analysing IoT applications. *J Syst Archit.* 2017;72:93–107. doi: 10.1016/j.sysarc.2016.06.008
13. Saleh N, Mashaly M. A dynamic simulation environment for container-based cloud data centers using ContainerCloudSim. In: 2019:332–336
14. Bouhouch L, Zbakh M, Tadonki C. DFMCloudsim: an extension of cloudsim for modeling and simulation of data fragments migration over distributed data centers. *International Journal of Computers and Applications.* 2024;46(1):1–20. doi: 10.1080/1206212X.2023.2277554
15. Bouhouch L, Zbakh M, Tadonki C. Data Migration: Cloudsim Extension. 2020:177–181. doi: 10.1145/3372454.3372472
16. Anand N, Saifulla M. EN-LAKP: lightweight authentication and key agreement protocol for emerging networks. *IEEE Access.* 2013;11:28645–28657. doi: 10.1109/ACCESS.2023.3255646
17. Anand N, Saifulla M, Ponnuru R, Alavalapati G, Patan R, Gandomi A. Securing software defined networks: a comprehensive analysis of approaches, applications, and future strategies against DoS attacks. *IEEE Access.* 2025;13:64473–64515. doi: 10.1109/ACCESS.2024.3520478
18. Alwasel K, Jha D, Hernandez E, et al. IOTSim-SDWAN: a simulation framework for interconnecting distributed datacenters over software-defined wide area network (SD-WAN). *J Parallel Distrib Comput.* 2020;143:17–35. doi: 10.1016/j.jpdc.2020.04.006
19. Peterson L, Davie B. *Computer networks: a systems approach*. Morgan Kaufmann. 5th ed., 2012.
20. Prasad R, Dovrolis C, Murray M, Claffy K. Bandwidth estimation: metrics, measurement techniques, and tools. *IEEE Netw.* 2003;17(6):27–35. doi: 10.1109/MNET.2003.1248658
21. Alsubai S, Garg H, Alqahtani A. A novel hybrid MSA-CSA algorithm for cloud computing task scheduling problems. *Symmetry.* 2023;15(10):1931. doi: 10.3390/sym15101931
22. Ramaswamy R, Weng N, Wolf T. Characterizing network processing delay. *IEEE Global Telecommunications Conference, 2004. GLOBECOM '04.*. 2004;3:1629-1634 Vol.3.
23. Abolfazli S, Sanaei Z, Alizadeh M, Gani A, Xia F. An experimental analysis on cloud-based mobile augmentation in mobile cloud computing. *IEEE Trans Consum Electron.* 2014;60(1):146–154. doi: 10.1109/TCE.2014.6780937
24. Masdari M, ValiKardan S, Shahi Z, Azar S. Towards workflow scheduling in cloud computing: a comprehensive analysis. *J Netw Comput Appl.* 2016;66:64–82. doi: 10.1016/j.jnca.2016.01.018
25. Uma K, Shukla S. Energy and performance-aware workflow scheduler using dynamic virtual network resource optimization under edge-cloud platform. *Comput Electr Eng.* 2025;123:110085. doi: 10.1016/j.compeleceng.2025.110085
26. Yassir S, Mostapha Z, Claude T. Workflow scheduling issues and techniques in cloud computing: a systematic literature review. In: Zbakh M, Essaaidi M, Manneback P, Rong C., eds. *Cloud Computing and Big Data: Technologies, Applications and Security*, , . 49 of *Lecture Notes in Networks and Systems*. Springer, 2019:213–228
27. Qoudhadh N, Zbakh M. Algorithms scheduling applied to microservices architectures: a survey study. In: 2023:1–12
28. Pandit R, Dwivedi M. Resources load sharing using RR, FCFS, SJF, MSJF, GP algorithms in cloud computing. *Neuroquantology.* 2022;20(11):7853–7872.
29. Anand N, Saifulla M. An efficient IDS for slow rate HTTP/2.0 DoS attacks using one class classification. In: 2023:1–9
30. Nemalikanti A, Kaki S, Ambati R, Tharun Y, Bagadi K, Ramesh D. Enhancing intrusion detection: protocol-based security using a hybrid RIDGE classifier on InSDN, UNSW-NB15, BoT-IoT, and ToN-IoT datasets. *Cluster Comput.* 2025;28:663–680. doi: 10.1007/s10586-025-05404-y

AUTHOR BIOGRAPHY

Professor **Mostapha Zbakh** is a Full Professor at ENSIAS, Mohammed V University in Rabat. He has published more than 66 articles in international journals and conference proceedings, accumulating over 760 citations (Google Scholar, 2025), with an h-index of 13 and an i10-index of 16. His research expertise lies in distributed and parallel systems, high-performance computing (HPC), grid and cloud computing, and intelligent data-center management. An active member of multiple research groups, he also organizes the international CloudTech Conference, which focuses on cloud, IoT, and software-defined networking (SDN) technologies.

Professor **Claude Tadonki** is a computer science researcher at Mines ParisTech – Centre de Recherche en Informatique in Paris. He has published extensively in international journals and conferences, with over 1,020 citations, an h-index of 17 and an i10-index of 25 (Google Scholar, 2025). His research interests include high-performance computing, cloud and distributed systems, optimization, algorithms, and operational research.

Professor **Yassir Samadi** is a Full Professor at the Faculty of Science of Tetouan. He has authored 17 publications. Which have accumulated over 5598 reads and more than 230 citations (ResearchGate, 2025). His research focuses on intelligent resource management in cloud platforms, benchmarking of big-data frameworks, and task-scheduling algorithms for heterogeneous computing infrastructures.

Nadia Qoudhadh is a Ph.D. candidate at the National School of Computer Science and Systems Analysis (ENSIAS), Mohammed V University in Rabat, Morocco. Her research focuses on cloud computing, software-defined networking (SDN), and cybersecurity. Her work centers on dynamic multi-objective scheduling in cloud-SDN environments, integrating bandwidth- and latency-aware resource allocation with QoS-driven orchestration.